

SuperMath

Roteiro de Apresentação — Computação Numérica

Ciência da Computação · UNISAGRADO · 2025

Este documento cobre tudo que o professor pode perguntar: o que cada método faz, como o sistema valida as entradas, o que acontece quando os dados estão errados, e a diferença entre os algoritmos.

1 VISÃO GERAL DO PROJETO

O SuperMath é uma aplicação web de métodos numéricos desenvolvida em Python com Streamlit. O objetivo é oferecer uma interface interativa para executar e visualizar algoritmos clássicos de cálculo numérico sem instalação local.

Python	Linguagem base de todo o sistema
Streamlit	Converte o script Python em aplicação web com interface gráfica
NumPy / SciPy	Operações matriciais e cálculos numéricos
SymPy	Cálculo simbólico — deriva funções automaticamente (Newton-Raphson)
Matplotlib	Geração dos gráficos de resultado
Groq API	Serviço de inferência que roda o modelo de IA (Llama 3.3 70B)

Por que Streamlit?

É o jeito mais rápido de publicar um script Python como app web sem precisar de HTML/CSS/JavaScript. Ideal para projetos acadêmicos e protótipos científicos.

2 MÓDULO — RAÍZES DE FUNÇÕES

Objetivo: encontrar o valor de x tal que $f(x) = 0$. O usuário digita a função em notação Python (ex.: $x^{**3} - x - 2$ representa $x^3 - x - 2$).

Como o sistema interpreta a função

O texto digitado passa pelo SymPy, que o converte em expressão simbólica. Se a sintaxe for inválida (ex.: parêntese faltando, operador errado), o sistema exibe uma mensagem de erro antes de tentar qualquer cálculo. Para o Newton-Raphson, o SymPy também deriva a função automaticamente.

2.1 Bisseção

Divide repetidamente o intervalo $[a, b]$ ao meio, ficando sempre com o subintervalo onde a função troca de sinal — até o intervalo ser menor que a tolerância definida.

a e b	Extremos do intervalo de busca
Tolerância	Tamanho mínimo do intervalo para parar (ex.: 1e-6)
Máx. iterações	Limite de divisões antes de encerrar forçado

O que acontece se os dados estiverem errados

- Se $f(a)$ e $f(b)$ tiverem o mesmo sinal, o Teorema de Bolzano não se aplica e o sistema retorna erro: "A função não muda de sinal no intervalo $[a, b]$." Nenhum cálculo é feito.
- Se a função não tiver raiz no intervalo mas trocar de sinal (ex.: assíntota), o método pode convergir para um ponto que não é raiz — por isso sempre confira o gráfico.

Por que é o mais lento?

Cada iteração reduz o intervalo à metade — convergência linear. Para uma tolerância de 10^{-6} num intervalo de tamanho 1, são necessárias aproximadamente 20 iterações. Sempre converge se o intervalo for válido.

2.2 Ponto Fixo

Reescreve $f(x) = 0$ como $x = g(x)$. A partir de um chute inicial x_0 , itera $x_{n+1} = g(x_n)$ até dois resultados consecutivos diferirem menos que a tolerância.

$g(x)$	Função de iteração — o usuário deve fornecer
x_0	Chute inicial

Tolerância	Diferença mínima entre iterações para parar
------------	---

O que acontece se os dados estiverem errados

- Se $|g'(x)| \geq 1$ na região da raiz, a sequência diverge — os valores crescem sem parar. O sistema detecta overflow e retorna: "Ocorreu um overflow. O método pode ter divergido."
- Se o método não convergir em max_iter iterações, retorna mensagem informando o limite atingido.
- Se $g(x)$ for uma função inválida, o SymPy retorna erro de sintaxe antes de começar.

■ A escolha de $g(x)$ é responsabilidade do usuário. A mesma equação pode ter múltiplas formas de $g(x)$, algumas convergentes e outras não.

2.3 Newton-Raphson

Usa a derivada de $f(x)$ para corrigir o ponto a cada iteração: $x_{n+1} = x_n - f(x_n) / f'(x_n)$. A derivada é calculada automaticamente pelo SymPy.

x_0	Chute inicial — quanto mais próximo da raiz, melhor
Tolerância	Diferença mínima entre iterações para parar

O que acontece se os dados estiverem errados

- Se $f'(x) = 0$ no caminho, há divisão por zero. O sistema verifica: se $|f'(x)| < 10^{-12}$, retorna "A derivada se aproximou de zero."
- Se x_0 estiver longe demais da raiz, o método pode divergir ou convergir para uma raiz errada (outro zero da função).
- Se a função não tiver derivada no ponto (ex.: $|x|$ em $x=0$), o SymPy pode não conseguir derivar e retorna erro.

Por que é o mais rápido?

Convergência quadrática: a cada iteração, o número de casas decimais corretas praticamente dobra. Para tolerância 10^{-12} , geralmente 5 a 10 iterações bastam.

2.4 Secantes

Aproxima a derivada usando dois pontos: $f'(x) \approx (f(x_1) - f(x_0)) / (x_1 - x_0)$. Dispensa o cálculo analítico da derivada.

x_0 e x_1	Dois chutes iniciais distintos
Tolerância	Diferença mínima entre iterações para parar

O que acontece se os dados estiverem errados

- Se $f(x_1) - f(x_0) \approx 0$ (pontos com o mesmo valor de f), há divisão por zero. O sistema verifica e retorna: "A diferença $f(x_1) - f(x_0)$ se aproximou de zero."
- Se $x_1 = x_0$, a divisão também falha pelo mesmo motivo.
- Pontos iniciais muito distantes da raiz podem causar divergência.

Diferença para o Newton-Raphson

O Secantes usa dois pontos para estimar a derivada em vez de calculá-la. É ligeiramente mais lento (convergência de ordem $\approx 1,618$ vs 2 do Newton), mas útil quando a derivada é difícil ou impossível de obter analiticamente.

2.5 Comparativo entre os métodos de raízes

Bisseção	Lenta (linear)	Não	Sim	Sim (se intervalo válido)
Ponto Fixo	Moderada	Não	Não	Depende de $g(x)$
Newton-Raphson	Rápida (quadrática)	Sim (automática)	Não	Depende de x_{n-1}
Secantes	Moderada-rápida	Não	Não	Depende de x_{n-1} e x_{n-2}

3 MÓDULO — SISTEMAS LINEARES

Objetivo: resolver um sistema $Ax = b$, onde A é a matriz de coeficientes ($n \times n$) e b é o vetor de termos independentes ($n \times 1$). O usuário informa a dimensão n , preenche A e b , e clica em RESOLVER.

Validação de entrada comum a todos os métodos

- Se a diagonal principal contiver um zero durante a eliminação, a matriz é considerada singular e o sistema não tem solução única — o sistema retorna None.
- Para os métodos iterativos, se a norma do vetor solução for muito grande, indica divergência.

3.1 Eliminação de Gauss com Pivoteamento Parcial

Método direto. Transforma A em triangular superior por operações de linha, depois resolve por substituição retroativa (de baixo para cima).

O que é pivoteamento parcial?

Antes de eliminar cada coluna, o algoritmo procura a linha com o maior valor absoluto naquela coluna e troca com a linha atual. Isso evita usar um número muito pequeno como divisor, o que amplificaria erros de arredondamento.

O que acontece se os dados estiverem errados

- Matriz singular (determinante = 0): o pivô chega a zero mesmo após as trocas. O sistema retorna None e exibe mensagem de erro.
- Sistema inconsistente (linhas proporcionais com b diferente): também resulta em pivô zero — tratado da mesma forma.
- Valores muito grandes em A ou b : o algoritmo ainda funciona, mas erros de ponto flutuante podem afetar a precisão.

3.2 Fatoração LU

Decompõe $A = P \cdot L \cdot U$, onde P é uma matriz de permutação, L é triangular inferior e U é triangular superior. O sistema é resolvido em duas etapas: $Ly = Pb$ (substituição progressiva) e depois $Ux = y$ (substituição retroativa). Usa `scipy.linalg` internamente.

Diferença para o Gauss

O Gauss modifica a matriz original e resolve diretamente. O LU guarda a decomposição separada de A — se precisar resolver o mesmo sistema com vários vetores b diferentes, a decomposição é feita só uma vez e reutilizada. Em termos de custo computacional para um único sistema, são equivalentes.

O que acontece se os dados estiverem errados

- Matriz singular: `scipy` lança `LinAlgError`, capturado pelo sistema, que retorna None, None, None e exibe mensagem de erro.
- Dimensão incorreta (A não quadrada): capturado como `ValueError`.

3.3 Jacobi

Método iterativo. Parte de um chute inicial $x = [0, 0, \dots, 0]$. A cada iteração, todas as variáveis são atualizadas simultaneamente usando apenas os valores da iteração anterior.

Fórmula de atualização

$$x_i^{(k+1)} = (b_i - \sum_{j \neq i} a_{ij} x_j^{(k)}) / a_{ii}$$

Condição de convergência

A matriz deve ser estritamente diagonalmente dominante: o valor absoluto do elemento da diagonal deve ser maior que a soma dos absolutos do restante da linha. O sistema emite aviso no terminal se isso não for satisfeito, mas não bloqueia a execução.

O que acontece se os dados estiverem errados

- Matriz não convergente: os valores de x explodem a cada iteração. O método encerra no limite de iterações e retorna o último resultado, que estará incorreto.
- Diagonal com zero: divisão por zero na fórmula — resulta em erro numérico.
- Tolerância muito alta (ex.: $1e-1$): o método para cedo com resultado impreciso.

3.4 Gauss-Seidel

Igual ao Jacobi, mas cada variável é atualizada imediatamente após ser calculada. Isso significa que x_i já usa o valor novo de $x_1, x_2, \dots, x_{i-1}$ dentro da mesma iteração.

Diferença para o Jacobi

No Jacobi, você guarda os valores antigos e atualiza todo mundo ao final de cada iteração. No Gauss-Seidel, você usa o valor recém-calculado de cada variável imediatamente — aproveita informação mais recente. Resultado: geralmente converge em menos iterações para a mesma tolerância.

O que acontece se os dados estiverem errados

Mesmos casos do Jacobi: matriz não convergente gera resultados incorretos ao fim das iterações; diagonal zero causa divisão por zero.

Gauss (pivoteamento)	Direto	Sim	Caso geral, uso único
Fatoração LU	Direto	Não (guarda L e U)	Múltiplos vetores b
Jacobi	Iterativo	Não	Sistemas grandes e esparsos
Gauss-Seidel	Iterativo	Não	Quando Jacobi converge lento

4 MÓDULO — AJUSTE DE CURVAS

Objetivo: dado um conjunto de pontos (x, y) , encontrar a curva que melhor representa os dados segundo o critério dos mínimos quadrados — minimiza a soma dos quadrados dos resíduos.

4.1 Regressão Linear

Ajusta uma reta $y = a \cdot x + b$ aos dados. Os coeficientes são calculados por fórmula fechada derivada das condições de mínimo da soma dos quadrados:

$$a = (n \cdot \sum xy - \sum x \cdot \sum y) / (n \cdot \sum x^2 - (\sum x)^2) \quad b = \bar{y} - a \cdot \bar{x}$$

O que acontece se os dados estiverem errados

- Se todos os pontos x forem iguais (variância zero), o denominador $n \cdot \sum x^2 - (\sum x)^2$ é zero — divisão por zero. O sistema retorna `None, None`.
- Com apenas 1 ponto: infinitas retas passam por um único ponto — resultado indefinido.
- Pontos com valores extremos (outliers): a reta se afasta da tendência real, mas o cálculo termina normalmente. O gráfico ajuda a identificar esse caso.

4.2 Mínimos Quadrados Polinomial

Ajusta um polinômio de grau n : $y = c_0 + c_1x + c_2x^2 + \dots + c_nx^n$. Internamente usa `np.polyfit`, que resolve um sistema linear de equações normais.

O que acontece se os dados estiverem errados

- Grau $\geq$ número de pontos: sistema subdeterminado — `np.polyfit` lança erro, capturado e retornado como `None`.
- Grau muito alto com poucos pontos: overfitting. O polinômio passa por todos os pontos mas oscila absurdamente entre eles. O cálculo termina, mas o modelo é inútil.
- Grau 1 é equivalente à Regressão Linear.

■ Regra prática: o grau do polinômio deve ser bem menor que o número de pontos. Grau 2 ou 3 já é suficiente para a maioria dos dados experimentais.

5 ASSISTENTE CLARK CALC

IA integrada ao SuperMath, alimentada pelo modelo Llama 3.3 70B via API do Groq. A diferença para um chatbot genérico é o **function calling**: o assistente tem acesso a uma função que lê os dados da sessão atual em tempo real (resultado, tabela de iterações, mensagens de erro), permitindo análises específicas.

Como acionar

- Botão "*Como funciona?*": gera explicação automática do método e resultado atual.
- Campo de texto na barra lateral: pergunta livre — pressione Enter para enviar.

O que ele consegue fazer

- Explicar passo a passo como o algoritmo chegou ao resultado exibido.
- Diagnosticar erros: se o cálculo falhou, explica o motivo específico.
- Responder dúvidas conceituais sobre qualquer método numérico.
- Analisar tabelas de iterações e identificar padrões de convergência.

Limitações

- Precisa de conexão com a internet (API externa).
- Sem a GROQ_API_KEY configurada, o assistente fica desativado.
- Não faz os cálculos — apenas analisa e explica o que o SuperMath calculou.

6 PERGUNTAS QUE O PROFESSOR PODE FAZER

■ Qual a diferença entre métodos diretos e iterativos?

Métodos diretos (Gauss, LU) chegam à solução em um número fixo de operações — não há iteração. Métodos iterativos (Jacobi, Gauss-Seidel) partem de um chute e refinam a solução até atingir uma tolerância. Diretos são exatos (sujeitos só a erros de ponto flutuante); iterativos são aproximados, mas escalam melhor para sistemas muito grandes.

■ Por que usar pivoteamento parcial no Gauss?

Sem pivoteamento, se o elemento diagonal for próximo de zero, dividimos por um número muito pequeno — isso amplifica drasticamente os erros de arredondamento do ponto flutuante. Trocar a linha pelo maior elemento disponível minimiza esse efeito.

■ Quando o Jacobi não converge?

Quando a matriz não é diagonalmente dominante, ou seja, quando o elemento da diagonal de alguma linha é menor ou igual à soma dos absolutos do resto da linha. Nesses casos, os valores iterados crescem sem limite.

■ Qual a diferença entre Jacobi e Gauss-Seidel?

No Jacobi, todos os x_i são atualizados com os valores da iteração anterior — as atualizações são independentes. No Gauss-Seidel, cada x_i é atualizado imediatamente usando os valores mais recentes já calculados na mesma iteração. O Gauss-Seidel geralmente converge mais rápido, mas os dois precisam da mesma condição de convergência.

■ Por que o Newton-Raphson é mais rápido?

Porque tem convergência quadrática: a cada iteração, o número de casas decimais corretas aproximadamente dobra. Os outros métodos (Bisseção, Ponto Fixo, Secantes) têm convergência linear ou superlinear — crescem mais devagar.

■ Quando usar Secantes em vez de Newton-Raphson?

Quando a função não tem derivada analítica fácil de obter, ou quando a derivada é computacionalmente cara. O Secantes aproxima a derivada por diferença finita — perde um pouco de velocidade, mas dispensa o cálculo simbólico.

■ O que é overfitting no ajuste de curvas?

Quando o grau do polinômio é alto demais para a quantidade de pontos, o polinômio passa exatamente por todos os pontos mas oscila absurdamente entre eles. Ele memoriza os dados em vez de generalizar a tendência — inútil para previsão.

■ Como o sistema sabe que um cálculo falhou?

Cada função retorna uma tupla (resultado, mensagem). Se resultado é None, houve falha. O Streamlit verifica isso antes de exibir e mostra a mensagem de erro no lugar do resultado. Para métodos iterativos, retornar o vetor após `max_iter` sem convergir é tratado como aviso, não erro fatal.

■ Por que usaram Streamlit e não Flask/Django?

Streamlit foi escolhido pela velocidade de desenvolvimento: com poucas linhas de Python já se tem formulários, gráficos e layout responsivo. Flask e Django requerem HTML, CSS e rotas adicionais — exagero para um projeto acadêmico de cálculo numérico.

■ O Clark Calc faz os cálculos numéricos?

Não. Ele é um modelo de linguagem que lê e explica os resultados — não executa algoritmos matemáticos. Os cálculos são feitos pelas rotinas Python (NumPy/SciPy/SymPy). O Clark Calc acessa esses resultados via function calling e os interpreta em linguagem natural.

7 RESUMO RÁPIDO — UMA LINHA POR MÉTODO

Bisseção	Divide o intervalo ao meio até achar a raiz
Ponto Fixo	Itera $x = g(x)$ até convergir — depende de $g(x)$ bem escolhida
Newton-Raphson	Usa a derivada para convergir rápido (quadrático)
Secantes	Newton sem derivada — usa dois pontos para estimá-la
Gauss c/ Pivoteamento	Triangulariza o sistema e resolve de trás pra frente
Fatoração LU	Decompõe $A = LU$ e resolve em duas substituições
Jacobi	Iterativo: atualiza tudo junto com valores da iteração anterior
Gauss-Seidel	Iterativo: usa valores recém-calculados dentro da mesma iteração
Regressão Linear	Ajusta a melhor reta $y = ax + b$ pelo critério dos mínimos quadrados
Mínimos Quadrados Polinomial	Ajusta o melhor polinômio de grau n aos dados

■ Dica final: execute pelo menos um cálculo de cada módulo antes da apresentação para ter os resultados na tela — facilita demonstrar o Clark Calc respondendo sobre um resultado real.